

1. cosign sign-blob artifact.md --bundle artifact.bundle

```
(.venv) (base) bezbakri@bezbakriacer:/mnt/c/MyFolder/school_trash/nyu/fall_2026/supply_chain_sec/assignment1/python-rekor-monitor-template$ cosign sign-blob artifact.md --bundle artifact.bundle.json && cat artifact.bundle.json | jq | tail -n6
Generating ephemeral keys...

The sigstore service, hosted by sigstore a Series of LF Projects, LLC, is provided pursuant to the Hosted Project Tools Terms of Use, available at https://lfprojects.org/policies/hosted-project-tools-terms-of-use/.
Note that if your submission includes personal data associated with this signed artifact, it will be part of an immutable record.
This may include the email address associated with the account with which you authenticate your contractual Agreement.
This information will be used for signing this artifact and will be stored in public transparency logs and cannot be removed later, and is subject to the Immutable Record notice at https://lfprojects.org/policies/hosted-project-tools-immutable-records/.

By typing 'y', you attest that (1) you are not submitting the personal data of any other person; and (2) you understand and agree to the statement and the Agreement terms at the URLs listed above.
Are you sure you would like to continue? [y/N] y
error opening browser: exec: "xdg-open,x-www-browser,www-browser": executable file not found in $PATH
Go to the following link in a browser:

Enter verification code:
Using payload from: artifact.md
Wrote bundle to file artifact.bundle.json
  "algorithm": "SHA2_256",
  "digest": "qWm3PZCkm8Le1SnWmxYFxbRkIblzz+yVjnaGB0FLWto="
  },
  "signature": "MEUCIAL2I03ksRQxydy8KtoqfWw06dHtwAGHolabuMayDUouAiEA1+3UTSzt/IKtjoDr4b90aUg0dXurKdUicUVZdsF/w="
}
```

2. main.py --inclusion <logIndex> --artifact artifact.md To verify that your log index is present in Rekor.

```
(base) bezbakri@bezbakriacer:/mnt/c/MyFolder/school_trash/nyu/fall_2026/supply_chain_sec/assignment1/python-rekor-monitor-template$ python3 main.py --inclusion 3042934851 --artifact artifact.md
Offline verification successful
```

3. main.py -c To fetch and store the latest checkpoint.

```
(base) bezbakri@bezbakriacer:/mnt/c/MyFolder/school_trash/nyu/fall_2026/supply_chain_sec/assignment1/python-rekor-monitor-template$ python3 main.py -c
{
  "inactiveShards": [
    {
      "rootHash": "4d006aa46efcb607dd51d900b1213754c50cc9251c3405c6c2561d9d6a2f3239",
      "signedTreeHead": "rekor.sigstore.dev - 3904496407287907110\n4163431\nTQBqpG78tgfdUdkAsSE3VMUMySUCNAXGwLYdnWovMjk=\n\n\u002014 rekor.sigstore.dev wNI9ajBFAiAs4BDVdu0LnYgGaeUoouH5j5GpaQcL2bRbww1Tw78xcTAiHAnrGbEQ7uXWdRVb1m/U1HtrRhKUSqW0xqsmrwAISLWjI\n",
      "treeID": "3904496407287907110",
      "treeSize": 4163431
    },
    {
      "rootHash": "2b9c578071e120aa84c9858783daa671565e4165f3a7dfa199aad81940299d8e",
      "signedTreeHead": "rekor.sigstore.dev - 2605736670972794746\n117740831\nnK5xXgHHhIKqEyYWHg9qmcVZeQWXzp9+hmarYGUApnY4=\n\n\u002014 rekor.sigstore.dev wNI9ajBFAiEAvrCF6vKVWLaJ9JnEN/3XjW2D+Lb4fswXT00DTbIHVgCIHKpmsHn9PL5f0fF+s015USmNk8sR9yaTYSkhQ7Z7QHe\n",
      "treeID": "2605736670972794746",
      "treeSize": 117740831
    }
  ],
  "rootHash": "98e20267fa5bb647560c8f4de800ffec5e651d78f2da9671787599dd2aa6565",
  "signedTreeHead": "rekor.sigstore.dev - 1193050959916656506\n2956703157\nnm0ICZ/pbtkdWDI9N6AD/70XmUdePLaInF4dZndKqZWU=\n\n\u002014 rekor.sigstore.dev wNI9ajBGAiEAItinvK8m8AH5KVNfIlN0Cj0u4RehcYZo6C7pYHz+w8ECIQccx4wVXgl+0UL1S6Yj2hF9CgxTU5IMwezOrhUAfmucAw=\n",
  "treeID": "1193050959916656506",
  "treeSize": 2956703157
}
```

4. main.py --consistency --tree-id <> --tree-size <> --root-hash <> To verify the consistency of the log between two stages.

```
(base) bezbakri@bezbakriacer:/mnt/c/MyFolder/school_trash/nyu/fall_2026/supply_chain_sec/assignment1/python-rekor-monitor-template$ python3 main.py --consistency --tree-id 1193050959916656506 --tree-size 5114308 --root-hash aa9f28964e1205576a05cbc6b66222124142b9fb15d803f55d8cbb1d74e766d7
Consistency verification successful
```

Problem 3:

1. a. In this case, there is no MODIFY on scripts/create_artifact.py, and we MATCH * with products from checkout in every step following the first. So verification against this attack would fail.
b. Verification would succeed because there's no step checking what happens before checkout on upstream. Verification against the commit history might help in this case
2. Verification would succeed, as with the developer key, you can sign link metadata for a create-artifact step where the artifact has malicious data. This can be solved with MFA.
3. In this case, verification would fail because the sign step MATCHes artifact.md with the products from create-artifact. If this step never ran, there is nothing to match against, and hence, verification fails.
4. Verification would fail as the metadata from running the steps again will not match with older metadata. So all the MATCH steps with artifact.md cannot link it to this week's create-artifact run.
5. Verification would fail as the sign step MATCHes artifact.md with the products from create-artifact. If artifact.md changed, then the MATCH fails, and hence, verification fails.
6. Verification would succeed. If the layout signing key is compromised, then the attacker could do whatever they want. MFA is the best way to defend against this.